IaC beyond Terraform

OpenTofu and ecosystem choices, Pulumi in real languages, Terragrunt structure, testing infrastructure code, policy as code and drift detection, and safe state migrations.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

Contents

1. OpenTofu and the Terraform ecosystem choices (12 min)
2. Pulumi: infrastructure in real programming languages (12 min)
3. Terragrunt, modules and multi-environment structure (12 min)
4. Testing infrastructure code (12 min)
5. Policy as code and drift detection (12 min)
6. State, secrets and safe migrations (12 min)
Cheat sheet

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 1. OpenTofu and the Terraform ecosystem choices

12 minutes. Understand how OpenTofu relates to Terraform, what compatibility to expect, and how to choose and migrate between tools with a safe plan.

Terraform and OpenTofu share a language and most providers, but they are separate projects with different licences and release paths. Knowing what is the same and what differs lets you choose on purpose.

What you will learn

- Why OpenTofu exists and who governs it
- What is compatible: HCL, providers and state
- What differs: features, registry and licence
- A safe migration procedure
- Pinning versions and keeping options open

Background

OpenTofu is an open source fork of Terraform, hosted by the Linux Foundation, started after Terraform moved to a source available licence. It keeps the HCL language and uses the same provider protocol, so most providers and modules work in both.

Compatibility

For a given version range, the state format and configuration are largely compatible, which makes switching practical. Features have diverged over time, so check the release notes of both tools. Test your own modules instead of trusting a general claim.

Differences

Examples are features added in only one project, different default registries and different licence terms for the tool itself. Providers are separate software with their own licences. Check how your vendor, your cloud provider and your compliance team treat each licence.

A safe migration

Back up the state, pin the current tool version, run a plan with the current tool and save the output. Then run init and plan with the new tool in a copy of the environment and compare. The plan should show no changes. Switch one stack at a time and keep a rollback by restoring the state backup.

Keep options open

Pin the required version in code, avoid tool specific features in shared modules and run your CI against the versions you support. This reduces the cost of changing later in either direction.

Diagram: OpenTofu and the Terraform ecosystem choices

Part	What it does	What goes wrong
HCL code	Same language.	Tool specific features.
Terraform or OpenTofu	Plans and applies.	Version drift.
Providers	Talk to APIs.	Version mismatch.
State	Records resources.	Corrupted or lost.

Flow: HCL code to Terraform or OpenTofu / Terraform or OpenTofu to Providers / Terraform or OpenTofu to State

MARGA by EknathaLabs marga.eknathalabs.com

Incident: A plan that wanted to replace a database

Incident: *After switching tools, the first plan showed a destroy and create of a production database.*

```
$ tofu plan -no-color | tail -4
  # aws_db_instance.main must be replaced
-/+ resource "aws_db_instance" "main" {
Plan: 1 to add, 0 to change, 1 to destroy.
(this is a simulated illustration, not a real incident)
```

The provider version was not pinned, so the new tool installed a newer provider that changed a default. The team stopped before applying, pinned the provider to the previous version, saw a clean plan and scheduled the provider upgrade as its own change.

Common mistakes

- Switching tools and upgrading providers in one step
- Not pinning provider and tool versions
- Applying a plan nobody read
- Assuming every feature exists in both tools
- No state backup before a migration

Check yourself

What do OpenTofu and Terraform share?

Answer: HCL and most provider compatibility

What should a migration plan show?

Answer: No changes

Why pin provider versions?

Answer: To avoid surprise changes

Where should you test first?

Answer: A copy of the environment

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. Pulumi: infrastructure in real programming languages

12 minutes. Describe infrastructure with Python, TypeScript or Go in Pulumi, understand stacks, outputs and secrets, and know when code beats configuration.

Pulumi uses general purpose languages to declare resources. You get loops, functions, types and tests, and you also get more ways to write unreadable infrastructure.

What you will learn

- The Pulumi model: programs, stacks and state
- Writing resources in Python
- Inputs, outputs and apply
- Secrets and configuration
- When code helps and when HCL is simpler

Model

A Pulumi program declares resources and the engine compares them with the state of a stack, then creates, updates or deletes what differs. A stack is one instance of the program, such as dev or prod, with its own configuration and state. State can live in the Pulumi service or in your own object storage.

Resources in code

A resource is an object created from a provider class. Because it is a normal program, you can use loops for many similar resources, functions for shared patterns and types to catch mistakes before a run.

Inputs and outputs

Some values are known only after a resource exists, so Pulumi wraps them as outputs. Use `apply` to transform an output, and pass outputs directly as inputs of other resources so the engine sees the dependency. Printing an output as a string before it resolves shows a placeholder, not the value.

Secrets and configuration

Mark values as secrets with `pulumi config set --secret` and the state stores them encrypted. Configuration is per stack, which keeps differences between environments in one place instead of copied files.

Code or configuration

Code shines for repeated patterns, validation and tests with ordinary test frameworks. It is a poor fit when the team lacks the language skills, or when plain declarative files would be clearer. Keep programs small, avoid network calls in code and review them like application code.

Diagram: Pulumi: infrastructure in real programming languages

Part	What it does	What goes wrong
Program	Declares resources.	Hidden logic.
Pulumi engine	Diffs against state.	Wrong stack.
Stack state	Encrypted secrets.	Lost backend.
Cloud API	Creates resources.	Permissions.

Flow: Program to Pulumi engine / Pulumi engine to Stack state / Pulumi engine to Cloud API

Incident: An output used before it existed

Incident: *A script printed a bucket name that never worked in a connection string.*

MARGA by EknathaLabs marga.eknathalabs.com

```
$ pulumi stack output bucketUrl
s3://app-assets-1f3a9
$ pulumi preview --diff | head -4
~ aws:s3/bucket:Bucket: (update)
    [id=app-assets-1f3a9]
(this is a simulated illustration, not a real incident)
```

The program built the URL by joining plain strings with the output object, which produced an unresolved placeholder in the application. Using apply to build the string after the value existed fixed it, and a unit test now checks the final value.

Common mistakes

- Treating outputs as plain strings
- One huge program for every environment
- Secrets in plain configuration
- Network calls and randomness inside programs
- No preview before an update

Check yourself

What is a Pulumi stack?

Answer: One instance of a program with its own config and state

Why are outputs special?

Answer: Their values are known only after creation

How do you store a secret?

Answer: Set it as a secret config value

When is code a poor fit?

Answer: When plain declarative files are clearer for the team

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 3. Terragrunt, modules and multi-environment structure

12 minutes. Structure infrastructure repositories with reusable modules, environment folders and Terragrunt, keeping configuration dry without hiding the blast radius.

Copying a Terraform folder for each environment works until the fifth copy. Good structure separates reusable modules from environment configuration and limits the size of each state.

What you will learn

- Modules versus environments
- Small states and blast radius
- What Terragrunt adds
- Dependencies between stacks
- Versioning modules and promoting changes

Modules and environments

A module is a reusable unit such as a network or a database. An environment folder calls modules with its own values. Modules live in a separate repository or directory and are versioned, so that production can use version 1.4 while dev tests 1.5.

Blast radius

One huge state means every plan reads everything and a mistake can touch everything. Split by lifecycle and risk: network, data, compute and applications each have their own state. A failure or a lock affects only that part.

Terragrunt

Terragrunt is a thin wrapper that generates backend and provider configuration, passes inputs from a hierarchy of files and runs several stacks in order. It removes repeated backend blocks and keeps the folder tree consistent. It also adds a layer that people must learn, so use it when repetition is a real problem.

Dependencies

A stack often needs outputs from another, such as the network id. Terragrunt dependency blocks read those outputs and define an order. Mock outputs let plans run before the dependency exists, but they must never be used in an apply.

Promotion

Change a module in a branch, release a tag, then update the version in dev, then staging and then production, reading the plan each time. A short, boring pipeline with a manual approval for production is better than a fast pipeline with surprises.

Diagram: Terragrunt, modules and multi-environment structure

Part	What it does	What goes wrong
Module repo	Versioned modules.	Unpinned refs.
Environment folders	Inputs per environment.	Copy and paste.
Terragrunt	Backends and order.	Hidden magic.
Separate states	Small blast radius.	One big state.

Flow: Module repo to Environment folders / Environment folders to Terragrunt / Terragrunt to Separate states

MARGA by EknathaLabs
marga.eknathalabs.com

Incident: A single apply that touched everything

Incident: *A change to a tag in a shared network stack planned changes for 400 resources.*

```
$ terragrunt run-all plan --terragrunt-non-interactive 2>&1 | grep -c "will be updated in-place"
412
$ git diff --stat | tail -1
 1 file changed, 1 insertion(+)
(this is a simulated illustration, not a real incident)
```

All environments pointed to the unversioned main branch of the module, so a one line change affected every stack at once. The team pinned module versions by tag, promoted them through dev and staging and split the network stack so a change touches fewer resources.

Common mistakes

- Pointing modules at a branch instead of a tag
- One state for the whole company
- Mock outputs used in an apply
- Copying folders for each environment
- No review of the plan for each environment

Check yourself

Why version modules?

Answer: So environments adopt changes on purpose

What limits blast radius?

Answer: Smaller states split by lifecycle

What does Terragrunt remove?

Answer: Repeated backend and provider configuration

When are mock outputs acceptable?

Answer: For plans before a dependency exists

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 4. Testing infrastructure code

12 minutes. Test infrastructure with formatting, validation, static checks, native tests and short lived real deployments, matched to the cost and risk of each layer.

Infrastructure changes are expensive to get wrong, yet often untested. A layered approach catches simple mistakes in seconds and reserves slow real deployments for what only they can prove.

What you will learn

- The testing pyramid for infrastructure
- Format, validate and lint
- Static security checks and policy
- Native tests and plan assertions
- Integration tests with real resources

The pyramid

Cheap fast checks run on every commit, medium checks run on every pull request and expensive tests run on a schedule or before a release. Spend most effort at the cheap end and add expensive tests for modules that many teams use.

Format, validate, lint

Use `fmt -check`, `validate` and a linter such as tflint to catch syntax errors, unused variables and invalid instance types. They need no credentials and finish in seconds.

Static security checks

Scan plans or code with tools such as Checkov, Trivy or a policy engine to find open security groups, unencrypted storage and public buckets before anything is created. Treat findings as review comments with clear fixes.

Native tests

Terraform and OpenTofu support test files with run blocks and assertions. A test can use a plan only to check expected values, or apply into a temporary environment and destroy afterward. Test module inputs and outputs, and the validation rules you wrote.

Integration tests

Tools such as Terratest deploy real resources, check them and destroy them. They give strong confidence but take minutes and cost money. Run them for shared modules, use a separate account with budget limits and make sure cleanup runs even when a test fails.

Diagram: Testing infrastructure code

Part	What it does	What goes wrong
Format and validate	Seconds.	Skipped locally.
Static checks	Security rules.	Ignored findings.
Plan and native tests	Assertions.	Weak assertions.
Real deployment tests	Real resources.	Orphaned resources.

Flow: Format and validate to Static checks / Static checks to Plan and native tests / Plan and native tests to Real deployment tests

MARGA by EknathaLabs marga.eknathalabs.com

Incident: A test environment left running for a month

Incident: *An integration test failed midway and left a database and a load balancer running.*

```
$ aws resourcegroupstaggingapi get-resources --tag-filters Key=env,Values=test --query 'ResourceTa \
    gMappingList[].ResourceARN' --output text | wc -w
14
$ aws ce get-cost-and-usage --help | head -1
GET-COST-AND-USAGE()
(this is a simulated illustration, not a real incident)
```

The test destroyed resources only on success. Cleanup now runs in a final step that always executes, tests tag everything with an owner and expiry, and a nightly job deletes anything past its expiry in the test account.

Common mistakes

- Running only manual plans
- Integration tests with no cleanup guarantee
- Tests that assert nothing
- Findings from scanners ignored for months
- Testing in the production account

Check yourself

Which checks need no credentials?

Answer: Format, validate and lint

What do native tests assert?

Answer: Expected values in a plan or created resources

What must integration tests do?

Answer: Always clean up, even on failure

What should static security checks find?

Answer: Open ports, unencrypted storage and public buckets before creation

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 5. Policy as code and drift detection

12 minutes. Enforce rules on plans with OPA or Sentinel style policies and detect changes made outside code, then decide whether to revert or adopt them.

Reviews miss things. Policy as code applies the same rules to every plan, and drift detection finds the changes people made by hand so the code and reality match again.

What you will learn

- What to enforce with policy
- Policies against plan output
- Where to run policy in the pipeline
- Detecting and classifying drift
- Reverting or adopting drift

What to enforce

Good policies are specific and explainable: storage must be encrypted, only approved regions and instance sizes are allowed, public access needs an approval tag, every resource has an owner tag and cost centre. Avoid broad rules that no one can satisfy.

Policies on a plan

A plan can be exported as JSON, and a policy engine such as Open Policy Agent with Conftest evaluates rules against it. Each rule returns a message that says what is wrong and how to fix it. Write tests for the policies themselves, with examples that pass and fail.

In the pipeline

Run policy on every pull request so the author sees failures early, and run it again before apply. Warnings for new rules and hard failures for established rules let teams adapt. An exemption process with an owner and an expiry avoids silent bypasses.

Detecting drift

Drift happens when someone changes a resource in the console or another tool. A scheduled plan, with no changes expected, reports differences. Classify drift: a harmless tag change, a risky security group change or an emergency fix that should become code.

Revert or adopt

If the change was wrong, apply the code to revert it and find out why access allowed it. If it was a valid emergency fix, copy it into code through a pull request so the next apply does not undo it. Reduce console write access to cut drift at the source.

Diagram: Policy as code and drift detection

Part	What it does	What goes wrong
Plan JSON	Proposed changes.	Unreviewed.
Policy engine	Evaluates rules.	Weak rules.
Apply	Creates changes.	Bypass.
Drift check	Scheduled plan.	Ignored report.

Flow: Plan JSON to Policy engine / Policy engine to Apply / Apply to Drift check

Incident: A security group opened in the console

Incident: *During an incident someone opened a port to the internet and never closed it.*

MARGA by EknathaLabs
marga.eknathalabs.com

```
$ terraform plan -detailed-exitcode -no-color | tail -5
  # aws_security_group.web will be updated in-place
  ~ ingress {
      + cidr_blocks = ["0.0.0.0/0"]
Plan: 0 to add, 1 to change, 0 to destroy.
(this is a simulated illustration, not a real incident)
```

The nightly drift plan reported the change and a ticket was opened automatically. The team reverted the rule through code, reviewed who could edit security groups in the console and added a policy that denies public ingress without an approved tag.

Common mistakes

- Policies nobody can explain
- Policies with no tests
- Exemptions without owner or expiry
- Never running a plan against reality
- Fixing drift by hand a second time

Check yourself

What does a policy engine read?

Answer: The plan output as data

What is drift?

Answer: Resources changed outside the code

How do you detect drift?

Answer: A scheduled plan expecting no changes

When should drift become code?

Answer: When it was a valid emergency fix

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. State, secrets and safe migrations

12 minutes. Protect state files and secrets, use remote backends with locking, and restructure code with moved, import and removed blocks without recreating resources.

State is the memory of your infrastructure tool, and it often contains secrets. Refactoring code without telling the tool what moved leads to destroy and create. Safe procedures avoid both problems.

What you will learn

- Remote state, locking and access control
- Secrets in state and in code
- Moving resources with moved blocks
- Importing existing resources
- Removing from state without destroying

Remote state

Store state in a remote backend such as an object store with versioning, encryption and a lock table, so only one apply runs at a time and you can roll back a bad write. Limit read access, because state may contain passwords and keys.

Secrets

Do not put secrets in code or variable files in the repository. Read them at run time from a secrets manager, mark outputs as sensitive and be aware that sensitive values are still stored in state. Prefer resources that generate and store secrets in the manager without exposing them to the tool.

Moved blocks

When you rename a resource or move it into a module, the tool sees a delete and a create. A `moved` block tells it the old address and the new address so the real resource is kept. Review the plan: it should show moves and no replacements.

Import

To bring an existing resource under management, write the resource block and use an `import` block or the import command with its id. Run a plan until it shows no changes, adjusting the code to match reality. Import in small steps and keep a backup of state.

Removing safely

To stop managing something without deleting it, use a `removed` block with `lifecycle { destroy = false }` or the state remove command. Use `prevent_destroy` on critical resources such as databases so a mistaken plan fails loudly.

Diagram: State, secrets and safe migrations

Part	What it does	What goes wrong
Refactored code	New addresses.	Renames.
Moved and import blocks	Map old to new.	Missing blocks.
Remote backend and lock	One apply at a time.	Stuck lock.
Real resources	Kept intact.	Recreated.

Flow: Refactored code to Moved and import blocks / Moved and import blocks to Remote backend and lock / Remote backend and lock to Real resources

MARGA by EknathaLabs marga.eknathalabs.com

Incident: A rename that planned to delete the database

Incident: *A refactor into modules changed resource addresses and the plan showed a destroy.*

```
$ terraform plan -no-color | grep -E "destroyed|created" | head -3
  # aws_db_instance.main will be destroyed
  # module.data.aws_db_instance.main will be created
Plan: 1 to add, 0 to change, 1 to destroy.
(this is a simulated illustration, not a real incident)
```

The code moved the resource into a module without a moved block. The team added a moved block from the old to the new address, the next plan showed only a move, and prevent_destroy was added to the database as a guard.

Common mistakes

- Renaming resources without moved blocks
- State in a local file or a repository
- Secrets in tfvars files
- Applying with no lock
- No prevent_destroy on databases

Check yourself

What does a moved block do?

Answer: Maps an old address to a new one so the resource is kept

Why use a lock?

Answer: Only one apply may change state at a time

Are sensitive outputs removed from state?

Answer: No, they are only hidden in output

How do you guard a database from deletion?

Answer: prevent_destroy in its lifecycle

MARGA by EknathaLabs
marga.eknathalabs.com

Cheat sheet: IaC beyond Terraform

Refactoring, checks and layout for infrastructure code.

Safe refactoring

Task	Tool
Rename a resource	`moved { from = A to = B }`
Adopt an existing resource	`import { to = ADDR id = ID }`
Move between states	`terraform state mv` after a backup
Remove from state only	`removed { from = ADDR lifecycle { destroy = false } }`

Checks

Check	Command
Validate	`tofu validate`
Plan exit code	`tofu plan -detailed-exitcode`
Test	`tofu test`
Policy scan	`checkov -d .`

Layout

Level	Holds
modules/	Versioned reusable code
envs/	Inputs per environment
state	One per environment and account

MARGA by EknathaLabs
marga.eknathalabs.com